Solution: Word Break

Problem Statement

Given a non-empty string and a dictionary containing a list of non-empty words, determine if the string can be segmented into a space-separated sequence of one or more dictionary words. Each word in the dictionary can be reused multiple times.

Examples

Example 1:

- Input:
  - String: "ilovecoding"
  - Dictionary: ["i", "love", "coding"]
- Expected Output: True
- Justification: The string can be segmented as "i love coding".

Example 2:

- Input:
  - String: "helloworld"
  - Dictionary: ["hello", "world", "hell", "low"]
- Expected Output: True
- Justification: The string can be segmented as "hello world".

Example 3:

- Input:
  - String: "enjoylife"
  - Dictionary: ["enj", "life", "joy"]
- Expected Output: False
- Justification: Despite having the words "enj" and "life" in the dictionary, we can't segment the string into the space-separated dictionary words.

Solution

Our algorithm's primary objective is to determine whether the given string can be broken down into a sequence of words present in the dictionary. To achieve this, we use a dynamic programming approach, maintaining an array to keep track of the possibility of forming valid sequences up to every index of the string. The primary idea is to iterate through the string and, at each step, check all possible word endings at the current position. If a valid word is found, and the starting position of that word was marked as achievable, we mark the current position as achievable too.

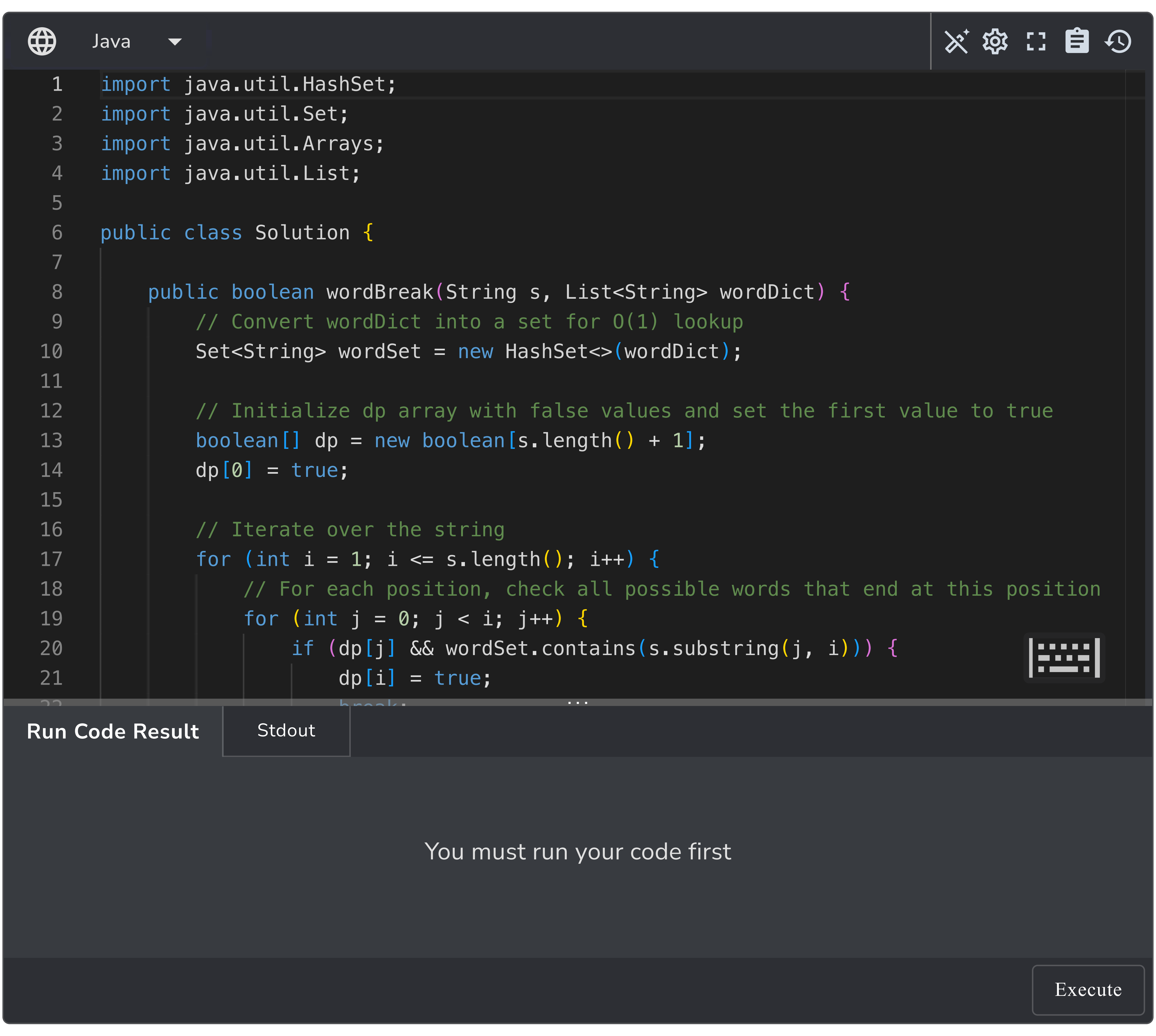
1. **Initialization**:
   - Begin by initializing a boolean array `dp` of size `n+1`, where `n` is the length of the string. This array will record whether the string can be segmented up to a certain index. We set the first element, `dp[0]`, to `true` since an empty string can always be segmented.
2. **Dynamic Programming**:
   - Iterate over the length of the string. For each index `i`, verify every substring ending at `i` and see if it exists in the dictionary.
   - If a valid word is found and the starting position (denoted as `dp[j]`) of the substring is true, set `dp[i+1]` to true.
   - Proceed in this manner until you reach the end of the string.
3. **Result**:
   - Once the iteration is complete, the value of `dp[n]` will indicate if the entire string can be segmented into dictionary words or not.
4. **Optimization**:
   - For faster lookups, convert the word dictionary into a set. This ensures constant time complexity when searching for words in the dictionary.

Algorithm Walkthrough

Given the string "helloworld" and dictionary ["hello", "world", "hell", "low"]:

- Initialize `dp` to `[true, false, false, ..., false]` (length = 11 since "helloworld" has 10 characters).
- For `i = 0`, substring = "h". It's not in the dictionary, so move to next.
- For `i = 1`, substring = "he", "h". Neither is in the dictionary.
- For `i = 4`, substring = "hello", which is in the dictionary and `dp[0]` is true. So, set `dp[5]` to true.
- Continuing this, when we get to `i = 9`, substring = "world" is in the dictionary, and `dp[5]` is true, so we set `dp[10]` to true.
- Finally, `dp[10]` is true, so "helloworld" can be segmented.

Code



```java
import java.util.HashSet;
import java.util.Set;
import java.util.Arrays;
import java.util.List;

public class Solution {

    public boolean wordBreak(String s, List<String> wordDict) {
        // Convert wordDict into a set for O(1) lookup
        Set<String> wordSet = new HashSet<>(wordDict);

        // Initialize dp array with false values and set the first value to true
        boolean[] dp = new boolean[s.length() + 1];
        dp[0] = true;

        // Iterate over the string
        for (int i = 1; i <= s.length(); i++) {
            // For each position, check all possible words that end at this position
            for (int j = 0; j < i; j++) {
                if (dp[j] && wordSet.contains(s.substring(j, i))) {
                    dp[i] = true;
```

Complexity Analysis

- **Time Complexity**: $O(n^2)$ due to the two nested loops where we check all possible substrings.
- **Space Complexity**: $O(n)$ for the DP array and the word set.

← Previous
Word Break (Medium)

Next →
Lowest Common Ancestor Of A Binary Search Tree (Medium)

☐ Mark as Completed